Honors Thesis:

Designing a Universal Information Language

Oscar Bender-Stone

March 26, 2026

Committee Members

Keith Kearnes
Mathematics
Thesis Advisor

Nathaniel Thiem
Mathematics
Honors Council Representative

Gowtham Kaki
Computer Science
Outside Reader

Agenda

Introduction

Language Overview

Syntax

Metatheory

Conclusion

Introduction

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Introduction

- Information helps us **store** and **organize** data

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Introduction

- Information helps us **store** and **organize** data
  - Throughout *all* history: so much data!
  - Want to actually use in, e.g., decisions
- Question: what *is* information?

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Introduction

- Information helps us **store** and **organize** data
  - Throughout *all* history: so much data!
  - Want to actually use in, e.g., decisions
- Question: what *is* information?
  - Depends on who you ask!

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Introduction

- Information helps us **store** and **organize** data
  - Throughout *all* history: so much data!
  - Want to actually use in, e.g., decisions
- Question: what *is* information?
  - Depends on who you ask!
  - **Ackoff:** Data-to-Wisdom Hierarchy[1]

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Introduction

- Information helps us **store** and **organize** data
  - Throughout *all* history: so much data!
  - Want to actually use in, e.g., decisions
- Question: what *is* information?
  - Depends on who you ask!
  - **Ackoff:** Data-to-Wisdom Hierarchy[1]
    - Data ⇒ Information ⇒ Wisdom

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Introduction

- Information helps us **store** and **organize** data
  - Throughout *all* history: so much data!
  - Want to actually use in, e.g., decisions
- Question: what *is* information?
  - Depends on who you ask!
  - **Ackoff:** Data-to-Wisdom Hierarchy[1]
    - Data ⇒ Information ⇒ Wisdom
  - **Buckland:** Based on usage[2]

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Introduction

- Information helps us **store** and **organize** data
  - Throughout *all* history: so much data!
  - Want to actually use in, e.g., decisions
- Question: what *is* information?
  - Depends on who you ask!
  - **Ackoff:** Data-to-Wisdom Hierarchy[1]
    - Data ⇒ Information ⇒ Wisdom
  - **Buckland:** Based on usage[2]
    - Can be a *process*, or *knowledge*
    - Most useful as *thing*
  - **Floridi:** Information is **truthful**[3]

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Challenges in Managing Information

Challenges in Managing Information

- **Depth:** domain specific data
  - Terminology, kinds of data, etc.

Challenges in Managing Information

- **Depth:** domain specific data
  - Terminology, kinds of data, etc.
  - Example: animal taxonomy
- **Breadth:** wide variety of domains

Challenges in Managing Information

- **Depth:** domain specific data
  - Terminology, kinds of data, etc.
  - Example: animal taxonomy
- **Breadth:** wide variety of domains
  - Different principles, methods

Challenges in Managing Information

- **Depth:** domain specific data
  - Terminology, kinds of data, etc.
  - Example: animal taxonomy
- **Breadth:** wide variety of domains
  - Different principles, methods
  - Example: Physics $\neq$ Biology!

Challenges in Managing Information

- **Depth:** domain specific data
  - Terminology, kinds of data, etc.
  - Example: animal taxonomy
- **Breadth:** wide variety of domains
  - Different principles, methods
  - Example: Physics $\neq$ Biology!
- **Standardization:** ambiguous interpretations

Challenges in Managing Information

- **Depth:** domain specific data
  - Terminology, kinds of data, etc.
  - Example: animal taxonomy
- **Breadth:** wide variety of domains
  - Different principles, methods
  - Example: Physics $\neq$ Biology!
- **Standardization:** ambiguous interpretations
  - Related issue: being **interoperable**

Challenges in Managing Information

- **Depth:** domain specific data
  - Terminology, kinds of data, etc.
  - Example: animal taxonomy
- **Breadth:** wide variety of domains
  - Different principles, methods
  - Example: Physics $\neq$ Biology!
- **Standardization:** ambiguous interpretations
  - Related issue: being **interoperable**
    - **Interoperable:** a portable data format

Challenges in Managing Information

- **Depth:** domain specific data
  - Terminology, kinds of data, etc.
  - Example: animal taxonomy
- **Breadth:** wide variety of domains
  - Different principles, methods
  - Example: Physics $\neq$ Biology!
- **Standardization:** ambiguous interpretations
  - Related issue: being **interoperable**
    - **Interoperable:** a portable data format
    - Example: U.S. Health records

Existing Solutions

[1] Hitzler et al. (2009). *OWL 2 Web Ontology Language Primer*. W3C.
[2] Robinson, Webber & Eifrem (2015). *Graph Databases* (2nd ed.). O'Reilly.
[3] Lenat (1995). *CYC: A large-scale investment in knowledge infrastructure*. CACM 38(11).

Existing Solutions

- **Resource Description Language (RDF):**[1]
  - Used in Web Ontology Language **(OWL)**

[1] Hitzler et al. (2009). *OWL 2 Web Ontology Language Primer*. W3C.
[2] Robinson, Webber & Eifrem (2015). *Graph Databases* (2nd ed.). O'Reilly.
[3] Lenat (1995). *CYC: A large-scale investment in knowledge infrastructure*. CACM 38(11).

Existing Solutions

- **Resource Description Language (RDF):**[1]
  - ▸ Used in Web Ontology Language **(OWL)**
  - \+ Built to be **interoperable**

[1] Hitzler et al. (2009). *OWL 2 Web Ontology Language Primer*. W3C.
[2] Robinson, Webber & Eifrem (2015). *Graph Databases* (2nd ed.). O'Reilly.
[3] Lenat (1995). *CYC: A large-scale investment in knowledge infrastructure*. CACM 38(11).

Existing Solutions

- **Resource Description Language (RDF):**[1]
  - ▸ Used in Web Ontology Language **(OWL)**
  - \+ Built to be **interoperable**
  - — No built-in logic engine

[1] Hitzler et al. (2009). *OWL 2 Web Ontology Language Primer*. W3C.
[2] Robinson, Webber & Eifrem (2015). *Graph Databases* (2nd ed.). O'Reilly.
[3] Lenat (1995). *CYC: A large-scale investment in knowledge infrastructure*. CACM 38(11).

Existing Solutions

- **Resource Description Language (RDF):**[1]
  - ▸ Used in Web Ontology Language **(OWL)**
  - \+ Built to be **interoperable**
  - — No built-in logic engine

[1] Hitzler et al. (2009). *OWL 2 Web Ontology Language Primer*. W3C.
[2] Robinson, Webber & Eifrem (2015). *Graph Databases* (2nd ed.). O'Reilly.
[3] Lenat (1995). *CYC: A large-scale investment in knowledge infrastructure*. CACM 38(11).

Existing Solutions

- **Resource Description Language (RDF):**[1]
  - ▸ Used in Web Ontology Language **(OWL)**
  - \+ Built to be **interoperable**
  - — No built-in logic engine
- **Labeled Property Graphs (LPGs):**[2]
  - ▸ Stores **properties** (node/edge metadata)

[1] Hitzler et al. (2009). *OWL 2 Web Ontology Language Primer*. W3C.
[2] Robinson, Webber & Eifrem (2015). *Graph Databases* (2nd ed.). O'Reilly.
[3] Lenat (1995). *CYC: A large-scale investment in knowledge infrastructure*. CACM 38(11).

Existing Solutions

- **Resource Description Language (RDF):**[1]
  - ▸ Used in Web Ontology Language **(OWL)**
  - \+ Built to be **interoperable**
  - — No built-in logic engine
- **Labeled Property Graphs (LPGs):**[2]
  - ▸ Stores **properties** (node/edge metadata)
  - \+ Efficient queries

[1] Hitzler et al. (2009). *OWL 2 Web Ontology Language Primer*. W3C.
[2] Robinson, Webber & Eifrem (2015). *Graph Databases* (2nd ed.). O'Reilly.
[3] Lenat (1995). *CYC: A large-scale investment in knowledge infrastructure*. CACM 38(11).

Existing Solutions

- **Resource Description Language (RDF):**[1]
  - ▸ Used in Web Ontology Language **(OWL)**
  - \+ Built to be **interoperable**
  - — No built-in logic engine
- **Labeled Property Graphs (LPGs):**[2]
  - ▸ Stores **properties** (node/edge metadata)
  - \+ Efficient queries
  - — No logic engine. Hacky tooling/scripts

[1] Hitzler et al. (2009). *OWL 2 Web Ontology Language Primer*. W3C.
[2] Robinson, Webber & Eifrem (2015). *Graph Databases* (2nd ed.). O'Reilly.
[3] Lenat (1995). *CYC: A large-scale investment in knowledge infrastructure*. CACM 38(11).

Existing Solutions

- **Resource Description Language (RDF):**[1]
  - ▸ Used in Web Ontology Language **(OWL)**
  - \+ Built to be **interoperable**
  - — No built-in logic engine
- **Labeled Property Graphs (LPGs):**[2]
  - ▸ Stores **properties** (node/edge metadata)
  - \+ Efficient queries
  - — No logic engine. Hacky tooling/scripts

[1] Hitzler et al. (2009). *OWL 2 Web Ontology Language Primer*. W3C.
[2] Robinson, Webber & Eifrem (2015). *Graph Databases* (2nd ed.). O'Reilly.
[3] Lenat (1995). *CYC: A large-scale investment in knowledge infrastructure*. CACM 38(11).

Existing Solutions

- **Resource Description Language (RDF):**[1]
  - ▸ Used in Web Ontology Language **(OWL)**
  - \+ Built to be **interoperable**
  - — No built-in logic engine
- **Labeled Property Graphs (LPGs):**[2]
  - ▸ Stores **properties** (node/edge metadata)
  - \+ Efficient queries
  - — No logic engine. Hacky tooling/scripts
- **Cyc:**[3]
  - ▸ Largest **knowledge base** to date

[1] Hitzler et al. (2009). *OWL 2 Web Ontology Language Primer*. W3C.
[2] Robinson, Webber & Eifrem (2015). *Graph Databases* (2nd ed.). O'Reilly.
[3] Lenat (1995). *CYC: A large-scale investment in knowledge infrastructure*. CACM 38(11).

Existing Solutions

- **Resource Description Language (RDF):**[1]
  - ▸ Used in Web Ontology Language **(OWL)**
  - \+ Built to be **interoperable**
  - — No built-in logic engine
- **Labeled Property Graphs (LPGs):**[2]
  - ▸ Stores **properties** (node/edge metadata)
  - \+ Efficient queries
  - — No logic engine. Hacky tooling/scripts
- **Cyc:**[3]
  - ▸ Largest **knowledge base** to date
  - \+ Sophisticated logic engine

[1] Hitzler et al. (2009). *OWL 2 Web Ontology Language Primer*. W3C.
[2] Robinson, Webber & Eifrem (2015). *Graph Databases* (2nd ed.). O'Reilly.
[3] Lenat (1995). *CYC: A large-scale investment in knowledge infrastructure*. CACM 38(11).

Existing Solutions

- **Resource Description Language (RDF):**[1]
  - ▸ Used in Web Ontology Language **(OWL)**
  - \+ Built to be **interoperable**
  - — No built-in logic engine
- **Labeled Property Graphs (LPGs):**[2]
  - ▸ Stores **properties** (node/edge metadata)
  - \+ Efficient queries
  - — No logic engine. Hacky tooling/scripts
- **Cyc:**[3]
  - ▸ Largest **knowledge base** to date
  - \+ Sophisticated logic engine
  - — Hard-coded, proprietary rules

[1] Hitzler et al. (2009). *OWL 2 Web Ontology Language Primer*. W3C.
[2] Robinson, Webber & Eifrem (2015). *Graph Databases* (2nd ed.). O'Reilly.
[3] Lenat (1995). *CYC: A large-scale investment in knowledge infrastructure*. CACM 38(11).

Existing Solutions

- **Resource Description Language (RDF):**[1]
  - ▸ Used in Web Ontology Language **(OWL)**
  - \+ Built to be **interoperable**
  - — No built-in logic engine
- **Labeled Property Graphs (LPGs):**[2]
  - ▸ Stores **properties** (node/edge metadata)
  - \+ Efficient queries
  - — No logic engine. Hacky tooling/scripts
- **Cyc:**[3]
  - ▸ Largest **knowledge base** to date
  - \+ Sophisticated logic engine
  - — Hard-coded, proprietary rules

[1] Hitzler et al. (2009). *OWL 2 Web Ontology Language Primer*. W3C.
[2] Robinson, Webber & Eifrem (2015). *Graph Databases* (2nd ed.). O'Reilly.
[3] Lenat (1995). *CYC: A large-scale investment in knowledge infrastructure*. CACM 38(11).

Proposed Solution

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

Proposed Solution

- **Welkin**: an information language

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

Proposed Solution

- **Welkin**: an information language
  - Pronounced: “VEL-KIN”

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

Proposed Solution

- **Welkin**: an information language
  - Pronounced: “VEL-KIN”
  - Old German word, meaning “cloud”, “sky”[1]

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

Proposed Solution

- **Welkin**: an information language
  - Pronounced: “VEL-KIN”
  - Old German word, meaning “cloud”, “sky”[1]
  - Represents *expansive* nature of language

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

Goals

- **Universality:** be completely expressive

Goals

- **Universality:** be completely expressive (**done**)

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size (**in progress**)

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size (**in progress**)
  - Be efficient

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size (**in progress**)
  - Be efficient
  - Can add optimizations *in* Welkin

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size (**in progress**)
  - Be efficient
  - Can add optimizations *in* Welkin
- **Standardization:** be completely *unambiguous*

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size (**in progress**)
  - Be efficient
  - Can add optimizations *in* Welkin
- **Standardization:** be completely *unambiguous* (**in progress**)

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size (**in progress**)
  - Be efficient
  - Can add optimizations *in* Welkin
- **Standardization:** be completely *unambiguous* (**in progress**)
  - Have a formal specification

Agenda

Introduction

Language Overview

Syntax

Metatheory

Conclusion

General Requirements

[1] Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2] Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

General Requirements

1. Express any computer program

[1] Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2] Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

General Requirements

1. Express any computer program
   - Upper bound: effective operations

[1] Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2] Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

General Requirements

1. Express any computer program
   - Upper bound: effective operations
   - => **Turing machines**[1]
2. Enable user-defined meaning

[1] Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2] Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

General Requirements

1. Express any computer program
   - Upper bound: effective operations
   - => **Turing machines**[1]
2. Enable user-defined meaning
   - Avoid meaningless symbols!

[1] Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2] Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

General Requirements

1. Express any computer program
   - Upper bound: effective operations
   - => **Turing machines**[1]
2. Enable user-defined meaning
   - Avoid meaningless symbols!
   - Symbol Grounding Problem[2]

[1] Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2] Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

General Requirements

1. Express any computer program
   - Upper bound: effective operations
   - => **Turing machines**[1]
2. Enable user-defined meaning
   - Avoid meaningless symbols!
   - Symbol Grounding Problem[2]
   - Treat user-defined denotations as free parameters

- Achieve both via **representations**

[1] Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2] Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

Representations

Representations

- Centered around *representing* things

Representations

- Centered around *representing* things
- $a - c \to b$: a **represents** b **in context** c

Representations

- Centered around *representing* things
- $a \mathrel{-\!\!-} c \to b$: a **represents** b **in context** c
  - a: **sign**

Representations

- Centered around *representing* things
- $a - c \to b$: a **represents** b **in context** c
  - a: **sign**
  - b: **referent**

Representations

- Centered around *representing* things
- $a - c \to b$: a **represents** b **in context** c
  - a: **sign**
  - b: **referent**
  - c: **context**

Representations

- Centered around *representing* things
- $a - c \to b$: a **represents** b **in context** c
  - a: **sign**
  - b: **referent**
  - c: **context**
- Track *faithfulness* of representations via **truth**

Units

Units

- Units are **finite programs**
- Built from **Handles**

Units

- Units are **finite programs**
- Built from **Handles**
  - Name origin: file/resource handles (programs)

Units

- Units are **finite programs**
- Built from **Handles**
  - Name origin: file/resource handles (programs)
  - Enables user-defined meaning

Units

- Units are **finite programs**
- Built from **Handles**
  - Name origin: file/resource handles (programs)
  - Enables user-defined meaning
  - User's goal: correctly specify!

Units

- Units are **finite programs**
- Built from **Handles**
  - Name origin: file/resource handles (programs)
  - Enables user-defined meaning
  - User's goal: correctly specify!
- Each unit contains its own **context** of representations
  - Written with braces $\{a, b, \ldots, z\}$

Units

- Units are **finite programs**
- Built from **Handles**
  - Name origin: file/resource handles (programs)
  - Enables user-defined meaning
  - User's goal: correctly specify!
- Each unit contains its own **context** of representations
  - Written with braces $\{a, b, \ldots, z\}$
  - Representations are *also* units

Information

Information

- Representations = **Truth**

Information

- Representations = **Truth**
- Information = **Proofs**

Information

- Representations = **Truth**
- Information = **Proofs**
  - Includes simple facts

Information

- Representations = **Truth**
- Information = **Proofs**
  - Includes simple facts
  - Expands to complex truth systems

Information

- Representations = **Truth**
- Information = **Proofs**
  - Includes simple facts
  - Expands to complex truth systems
- Major result: can express *any* "reliable" proof

Information

- Representations = **Truth**
- Information = **Proofs**
  - Includes simple facts
  - Expands to complex truth systems
- Major result: can express *any* "reliable" proof
  - More details later! (Metatheory)

Rules

- Three categories:

Rules

- Three categories:
  - **Primary:**

Rules

- Three categories:
  - **Primary:**
    - Transitivity in a context

Rules

- Three categories:
  - **Primary:**
    - Transitivity in a context
    - Congruence (build new units)

Rules

- Three categories:
  - **Primary:**
    - Transitivity in a context
    - Congruence (build new units)
    - Contextual lifting (use *between* contexts)
  - **Empty**

Rules

- Three categories:
  - **Primary:**
    - Transitivity in a context
    - Congruence (build new units)
    - Contextual lifting (use *between* contexts)
  - **Empty**
    - Defines empty unit

Rules

- Three categories:
  - **Primary:**
    - Transitivity in a context
    - Congruence (build new units)
    - Contextual lifting (use *between* contexts)
  - **Empty**
    - Defines empty unit
    - Enables exclusions

Rules

- Three categories:
  - **Primary:**
    - Transitivity in a context
    - Congruence (build new units)
    - Contextual lifting (use *between* contexts)
  - **Empty**
    - Defines empty unit
    - Enables exclusions
  - **Lattice**

Rules

- Three categories:
  - **Primary:**
    - Transitivity in a context
    - Congruence (build new units)
    - Contextual lifting (use *between* contexts)
  - **Empty**
    - Defines empty unit
    - Enables exclusions
  - **Lattice**
    - Makes units position independent

Rules

- Three categories:
  - **Primary:**
    - Transitivity in a context
    - Congruence (build new units)
    - Contextual lifting (use *between* contexts)
  - **Empty**
    - Defines empty unit
    - Enables exclusions
  - **Lattice**
    - Makes units position independent
- For details: consult thesis

Turing Completeness

Turing Completeness

- Part of Universality

Turing Completeness

- Part of Universality
- **Theorem:** *every unit corresponds to some program, and vice versa.*

Turing Completeness

- Part of Universality
- **Theorem:** *every unit corresponds to some program, and vice versa.*
  - For details: consult thesis
- Later:

Turing Completeness

- Part of Universality
- **Theorem:** *every unit corresponds to some program, and vice versa.*
  - For details: consult thesis
- Later: completeness for **proofs**!

Agenda

Introduction

Language Overview

Syntax

Metatheory

Conclusion

Syntax

- Two main parts

Syntax

- Two main parts
  - Encoding: numerical codes of characters

Syntax

- Two main parts
  - Encoding: numerical codes of characters
    - Building blocks of strings

Syntax

- Two main parts
  - Encoding: numerical codes of characters
    - Building blocks of strings
  - Grammar: strings allowed by language
    - Punctuation
    - How strings are combined

Encoding

- Characters written in `type-writer font`[1]

[1] Font used: Intel One Mono (https://github.com/intel/intel-one-mono), licensed under OFL 1.1.

Encoding

- Characters written in `type-writer font`[1]
- Define US-ASCII: Printable (codes: 32-126) + `EOF` (code: 127)

[1] Font used: Intel One Mono (https://github.com/intel/intel-one-mono), licensed under OFL 1.1.

Encoding

- Characters written in `type-writer font`[1]
- Define US-ASCII: Printable (codes: 32-126) + `EOF` (code: 127)
- Character classes: refer to thesis

[1] Font used: Intel One Mono (https://github.com/intel/intel-one-mono), licensed under OFL 1.1.

US-ASCII

Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph
32	20	Space	46	2E	.	60	3C	<	74	4A	J	88	58	X	102	66	f	116	74	t
33	21	!	47	2F	/	61	3D	=	75	4B	K	89	59	Y	103	67	g	117	75	u
34	22	"	48	30	0	62	3E	>	76	4C	L	90	5A	Z	104	68	h	118	76	v
35	23	#	49	31	1	63	3F	?	77	4D	M	91	5B	[	105	69	i	119	77	w
36	24	$	50	32	2	64	40	@	78	4E	N	92	5C	\	106	6A	j	120	78	x
37	25	%	51	33	3	65	41	A	79	4F	O	93	5D	]	107	6B	k	121	79	y
38	26	&	52	34	4	66	42	B	80	50	P	94	5E	^	108	6C	l	122	7A	z
39	27	'	53	35	5	67	43	C	81	51	Q	95	5F	_	109	6D	m	123	7B	{
40	28	(	54	36	6	68	44	D	82	52	R	96	60	`	110	6E	n	124	7C	|
41	29	)	55	37	7	69	45	E	83	53	S	97	61	a	111	6F	o	125	7D	}
42	2A	*	56	38	8	70	46	F	84	54	T	98	62	b	112	70	p	126	7E	~
43	2B	+	57	39	9	71	47	G	85	55	U	99	63	c	113	71	q			
44	2C	,	58	3A	:	72	48	H	86	56	V	100	64	d	114	72	r			
45	2D	-	59	3B	;	73	49	I	87	57	W	101	65	e	115	73	s			

Table 1: Printable US-ASCII codes and glyphs.

Grammar Approach

- Based on two papers

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

Grammar Approach

- Based on two papers
  - First: **Invertible Syntax Descriptions**[1]

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

Grammar Approach

- Based on two papers
  - First: **Invertible Syntax Descriptions**[1]
    - Processes strings, forwards and back

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

Grammar Approach

- Based on two papers
  - First: **Invertible Syntax Descriptions**[1]
    - Processes strings, forwards and back
  - Second: Edelmann et. al.[2]

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

Grammar Approach

- Based on two papers
  - First: **Invertible Syntax Descriptions**[1]
    - Processes strings, forwards and back
  - Second: Edelmann et. al.[2]
    - Used in proof of unambiguity

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

Grammar Approach

- Based on two papers
  - First: **Invertible Syntax Descriptions**[1]
    - Processes strings, forwards and back
  - Second: Edelmann et. al.[2]
    - Used in proof of unambiguity
    - Also ensures efficiency

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

Grammar Approach

- Based on two papers
  - First: **Invertible Syntax Descriptions**[1]
    - Processes strings, forwards and back
  - Second: Edelmann et. al.[2]
    - Used in proof of unambiguity
    - Also ensures efficiency
  - For details: refer to thesis

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

Grammar Approach

- Based on two papers
  - First: **Invertible Syntax Descriptions**[1]
    - Processes strings, forwards and back
  - Second: Edelmann et. al.[2]
    - Used in proof of unambiguity
    - Also ensures efficiency
  - For details: refer to thesis
- **Notes**:

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

Grammar Approach

- Based on two papers
  - First: **Invertible Syntax Descriptions**[1]
    - Processes strings, forwards and back
  - Second: Edelmann et. al.[2]
    - Used in proof of unambiguity
    - Also ensures efficiency
  - For details: refer to thesis
- **Notes**:
  - Current proofs are done

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

Grammar Approach

- Based on two papers
  - First: **Invertible Syntax Descriptions**[1]
    - Processes strings, forwards and back
  - Second: Edelmann et. al.[2]
    - Used in proof of unambiguity
    - Also ensures efficiency
  - For details: refer to thesis
- **Notes**:
  - Current proofs are done
  - Grammar is **not** finalized

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

Full Grammar

```
start := units - lexeme -> EOF,
units := "" | {unit - lexeme -> *{"",
"," - lexeme -> units}},
unit := node - lexeme -> *{
  ":=" - lexeme -> choices,
  *{"", arc - lexeme -> chain}
},
choices := *{"", "|"} - lexeme ->
choices_list,
choices_list := *{"",
  chain - lexeme -> *{"", "|" - lexeme
-> choice_list}
},
chain := *{"", "*"} - seq -> path -
lexeme -> *{"", arc - lexeme ->
chain},
arc := right_arc | other_arc,
right_arc := "-" - seq -> node - seq -
> "->",
other_arc := "<-" - seq -> node - seq
-> *{"-", "->"},
node := *{"", "*"} - seq -> path,
path := *{"", modifiers} - seq ->
trailer - seq -> *{"", graph},
modifiers := *{
  "@",
  "~" - seq -> *{"", "@"}
},
trailer := segment - seq -> *{"", "."
- seq -> trailer},
segment := HANDLE - lexeme -> *{"",
graph},
graph :=  "{" - lexeme -> units -
lexeme -> "}",
HANDLE := ID | STRING,
STRING := SQ_STRING | DQ_STRING,
SQ_STRING := "'" - seq -> *{SQ_CHAR,
"\'"} - seq_many_till -> "'",
DQ_STRING := '"' - seq -> *{DQ_CHAR,
'\"'} - seq_many_till -> '"',
ID := ID_CHAR - seq_many_till ->
*{RESERVED, WHITESPACE}
```

Listing 1: Full Welkin grammar.

Agenda

Introduction

Language Overview

Syntax

Metatheory

Conclusion

Formal System Limits

- Many theorems on limits:

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limits:
  - **Gödel incompleteness**

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limits:
  - **Gödel incompleteness**
    - **First:** PA can't prove everything

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limits:
  - **Gödel incompleteness**
    - **First:** PA can't prove everything
    - **Second**: PA cannot prove *own* consistency

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limits:
  - **Gödel incompleteness**
    - **First:** PA can't prove everything
    - **Second**: PA cannot prove *own* consistency
  - Etc.

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limits:
  - **Gödel incompleteness**
    - **First:** PA can't prove everything
    - **Second**: PA cannot prove *own* consistency
  - Etc.
- Usual approach: *chain of stronger theories*

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limits:
  - **Gödel incompleteness**
    - **First:** PA can't prove everything
    - **Second**: PA cannot prove *own* consistency
  - Etc.
- Usual approach: *chain of stronger theories*
  - Extend by $\mathrm{Con}(T)$ (encoding of consistency)

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limits:
  - **Gödel incompleteness**
    - **First:** PA can't prove everything
    - **Second**: PA cannot prove *own* consistency
  - Etc.
- Usual approach: *chain of stronger theories*
  - Extend by $\mathrm{Con}(T)$ (encoding of consistency)
  - Technically: this is enough, WITH non-constructive methods[1]

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limits:
  - **Gödel incompleteness**
    - **First:** PA can't prove everything
    - **Second**: PA cannot prove *own* consistency
  - Etc.
- Usual approach: *chain of stronger theories*
  - Extend by $\mathrm{Con}(T)$ (encoding of consistency)
  - Technically: this is enough, WITH non-constructive methods[1]
  - Problem: want to identify + trust proofs!

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Artëmov's Selector Proofs

[1] Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm[1]

[1] Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm[1]
  - Early Mentions (No Framework): Hilbert, Gödel, etc.

[1] Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm[1]
  - Early Mentions (No Framework): Hilbert, Gödel, etc.
  - Proves a weaker property: **serial-consistency**
- Approach: over a theory T, build a computable function *over* T-proofs

[1] Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm[1]
  - Early Mentions (No Framework): Hilbert, Gödel, etc.
  - Proves a weaker property: **serial-consistency**
- Approach: over a theory T, build a computable function *over* T-proofs
  1. Enumerate through all T-proofs in a meta-theory (e.g., PA)

[1] Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm[1]
  - Early Mentions (No Framework): Hilbert, Gödel, etc.
  - Proves a weaker property: **serial-consistency**
- Approach: over a theory T, build a computable function *over* T-proofs
  1. Enumerate through all T-proofs in a meta-theory (e.g., PA)
  2. Using T, show each T-proof p is consistent (no $\perp$)

[1] Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm[1]
  - Early Mentions (No Framework): Hilbert, Gödel, etc.
  - Proves a weaker property: **serial-consistency**
- Approach: over a theory T, build a computable function *over* T-proofs
  1. Enumerate through all T-proofs in a meta-theory (e.g., PA)
  2. Using T, show each T-proof p is consistent (no $\perp$)
- This works...*almost*

[1] Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm[1]
  - Early Mentions (No Framework): Hilbert, Gödel, etc.
  - Proves a weaker property: **serial-consistency**
- Approach: over a theory T, build a computable function *over* T-proofs
  1. Enumerate through all T-proofs in a meta-theory (e.g., PA)
  2. Using T, show each T-proof p is consistent (no $\perp$)
- This works...*almost*
  - Any inconsistency is *finite*
  - Method above aims to show *no* such example exists

[1] Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm[1]
  - Early Mentions (No Framework): Hilbert, Gödel, etc.
  - Proves a weaker property: **serial-consistency**
- Approach: over a theory T, build a computable function *over* T-proofs
  1. Enumerate through all T-proofs in a meta-theory (e.g., PA)
  2. Using T, show each T-proof p is consistent (no $\perp$)
- This works...*almost*
  - Any inconsistency is *finite*
  - Method above aims to show *no* such example exists
  - Problem: could use $\perp$ to prove this!

[1] Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm[1]
  - Early Mentions (No Framework): Hilbert, Gödel, etc.
  - Proves a weaker property: **serial-consistency**
- Approach: over a theory T, build a computable function *over* T-proofs
  1. Enumerate through all T-proofs in a meta-theory (e.g., PA)
  2. Using T, show each T-proof p is consistent (no $\perp$)
- This works...*almost*
  - Any inconsistency is *finite*
  - Method above aims to show *no* such example exists
  - Problem: could use $\perp$ to prove this!
  - Need to add restriction: **must be constructive**

[1] Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Serial-Soundness

[1] Consult: Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Serial-Soundness

- Extend Artëmov to **serial-soundness:**

[1] Consult: Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Serial-Soundness

- Extend Artëmov to **serial-soundness:**
  - Secret: *same* technique![1]

[1] Consult: Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Serial-Soundness

- Extend Artëmov to **serial-soundness:**
  - Secret: *same* technique![1]
- Let T be a first-order, RE theory

[1] Consult: Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Serial-Soundness

- Extend Artëmov to **serial-soundness:**
  - Secret: *same* technique![1]
- Let T be a first-order, RE theory
- T is **serial-sound**, if for some total function s:

[1] Consult: Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Serial-Soundness

- Extend Artëmov to **serial-soundness:**
  - Secret: *same* technique![1]
- Let T be a first-order, RE theory
- T is **serial-sound**, if for some total function s:
  - Accepts all proofs + preserves truth

[1] Consult: Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Serial-Soundness

- Extend Artëmov to **serial-soundness:**
  - Secret: *same* technique![1]
- Let T be a first-order, RE theory
- T is **serial-sound**, if for some total function s:
  - Accepts all proofs + preserves truth
  - Correctness *cannot* be proven with:

[1] Consult: Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Serial-Soundness

- Extend Artëmov to **serial-soundness:**
  - Secret: *same* technique![1]
- Let T be a first-order, RE theory
- T is **serial-sound**, if for some total function s:
  - Accepts all proofs + preserves truth
  - Correctness *cannot* be proven with:
    - Excluded Middle: $\varphi \vee \neg\varphi$

[1] Consult: Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Serial-Soundness

- Extend Artëmov to **serial-soundness:**
  - Secret: *same* technique![1]
- Let T be a first-order, RE theory
- T is **serial-sound**, if for some total function s:
  - Accepts all proofs + preserves truth
  - Correctness *cannot* be proven with:
    - Excluded Middle: $\varphi \vee \neg\varphi$
    - Explosion: $\bot \Rightarrow \varphi$

[1] Consult: Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Serial-Soundness

- Extend Artëmov to **serial-soundness:**
  - Secret: *same* technique![1]
- Let T be a first-order, RE theory
- T is **serial-sound**, if for some total function s:
  - Accepts all proofs + preserves truth
  - Correctness *cannot* be proven with:
    - Excluded Middle: $\varphi \vee \neg\varphi$
    - Explosion: $\bot \Rightarrow \varphi$
- T is **self-verifying** if, *in* PA, T can prove its own serial-soundness

[1] Consult: Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Serial-Soundness

- Extend Artëmov to **serial-soundness:**
  - Secret: *same* technique![1]
- Let T be a first-order, RE theory
- T is **serial-sound**, if for some total function s:
  - Accepts all proofs + preserves truth
  - Correctness *cannot* be proven with:
    - Excluded Middle: $\varphi \vee \neg\varphi$
    - Explosion: $\bot \Rightarrow \varphi$
- T is **self-verifying** if, *in* PA, T can prove its own serial-soundness
  - Notation: $\mathrm{PA} \vdash_T \varphi$ if $T \vdash \varphi$ and T self-verifying

[1] Consult: Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Reliable Proofs

- How to generalize to *any* theory?

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory
    - Can't *fully* trust b/c Russell's Paradox

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory
    - Can't *fully* trust b/c Russell's Paradox
    - But can trust, e.g., $A \subseteq B \Leftrightarrow A \cup B = B$

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory
    - Can't *fully* trust b/c Russell's Paradox
    - But can trust, e.g., $A \subseteq B \Leftrightarrow A \cup B = B$
    - Idea: check for some sound *restricted* theory
- Let T' be an RE theory, φ a sentence

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory
    - Can't *fully* trust b/c Russell's Paradox
    - But can trust, e.g., $A \subseteq B \Leftrightarrow A \cup B = B$
    - Idea: check for some sound *restricted* theory
- Let T' be an RE theory, φ a sentence
- **Reliable proof**: a triple (S, p_S, p_φ) s.t:
  - $S \subseteq_{\mathrm{RE}} T'$, $S \neq \varnothing$

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory
    - Can't *fully* trust b/c Russell's Paradox
    - But can trust, e.g., $A \subseteq B \Leftrightarrow A \cup B = B$
    - Idea: check for some sound *restricted* theory
- Let T' be an RE theory, φ a sentence
- **Reliable proof**: a triple (S, p_S, p_φ) s.t:
  - $S \subseteq_{\mathrm{RE}} T'$, $S \neq \varnothing$
  - p_S: constructs some self-verifying $S' \supseteq S$

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory
    - Can't *fully* trust b/c Russell's Paradox
    - But can trust, e.g., $A \subseteq B \Leftrightarrow A \cup B = B$
    - Idea: check for some sound *restricted* theory
- Let T' be an RE theory, φ a sentence
- **Reliable proof**: a triple (S, p_S, p_φ) s.t:
  - $S \subseteq_{\mathrm{RE}} T'$, $S \neq \varnothing$
  - p_S: constructs some self-verifying $S' \supseteq S$
  - p_φ: S-proof of φ

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory
    - Can't *fully* trust b/c Russell's Paradox
    - But can trust, e.g., $A \subseteq B \Leftrightarrow A \cup B = B$
    - Idea: check for some sound *restricted* theory
- Let T' be an RE theory, φ a sentence
- **Reliable proof**: a triple (S, p_S, p_φ) s.t:
  - $S \subseteq_{\mathrm{RE}} T'$, $S \neq \varnothing$
  - p_S: constructs some self-verifying $S' \supseteq S$
  - p_φ: S-proof of φ
- **Note:** needs contextual lifting!

Proof Completeness

- **Note:** still has incompleteness!

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited (Not surprising)

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited (Not surprising)
  - Also: not a *single* theory suffices

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited (Not surprising)
  - Also: not a *single* theory suffices
  - However, a theory *can* be self-verifying!

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited (Not surprising)
  - Also: not a *single* theory suffices
  - However, a theory *can* be self-verifying!
- Proof completeness = reaching limit *on* proofs

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited (Not surprising)
  - Also: not a *single* theory suffices
  - However, a theory *can* be self-verifying!
- Proof completeness = reaching limit *on* proofs
- How to get proof completeness?

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited (Not surprising)
  - Also: not a *single* theory suffices
  - However, a theory *can* be self-verifying!
- Proof completeness = reaching limit *on* proofs
- How to get proof completeness?
  - Basic idea: ZFC is self-verifying!

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited (Not surprising)
  - Also: not a *single* theory suffices
  - However, a theory *can* be self-verifying!
- Proof completeness = reaching limit *on* proofs
- How to get proof completeness?
  - Basic idea: ZFC is self-verifying!
    - Again, same proof from Artëmov!

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited (Not surprising)
  - Also: not a *single* theory suffices
  - However, a theory *can* be self-verifying!
- Proof completeness = reaching limit *on* proofs
- How to get proof completeness?
  - Basic idea: ZFC is self-verifying!
    - Again, same proof from Artëmov!
  - **Huge** leap in power; how to extend it?

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited (Not surprising)
  - Also: not a *single* theory suffices
  - However, a theory *can* be self-verifying!
- Proof completeness = reaching limit *on* proofs
- How to get proof completeness?
  - Basic idea: ZFC is self-verifying!
    - Again, same proof from Artëmov!
  - **Huge** leap in power; how to extend it?
  - Answer: comprehension!

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited (Not surprising)
  - Also: not a *single* theory suffices
  - However, a theory *can* be self-verifying!
- Proof completeness = reaching limit *on* proofs
- How to get proof completeness?
  - Basic idea: ZFC is self-verifying!
    - Again, same proof from Artëmov!
  - **Huge** leap in power; how to extend it?
  - Answer: comprehension!
  - Details in thesis

Agenda

Introduction

Language Overview

Syntax

Metatheory

Conclusion

Conclusion

- Welkin is a universal information language

Conclusion

- Welkin is a universal information language
  - Simple Rules

Conclusion

- Welkin is a universal information language
  - Simple Rules
  - Formalized Syntax

Conclusion

- Welkin is a universal information language
  - Simple Rules
  - Formalized Syntax
  - Robust Metatheory

Conclusion

- Welkin is a universal information language
  - Simple Rules
  - Formalized Syntax
  - Robust Metatheory
- Goals:

Conclusion

- Welkin is a universal information language
  - Simple Rules
  - Formalized Syntax
  - Robust Metatheory
- Goals:
  - Universality (**done**)

Conclusion

- Welkin is a universal information language
  - Simple Rules
  - Formalized Syntax
  - Robust Metatheory
- Goals:
  - Universality (**done**)
  - Scalability (**in progress**)

Conclusion

- Welkin is a universal information language
  - Simple Rules
  - Formalized Syntax
  - Robust Metatheory
- Goals:
  - Universality (**done**)
  - Scalability (**in progress**)
  - Standardization (**in progress**)

Comparisons to Existing Solutions

Comparisons to Existing Solutions

- **Resource Description Framework:**

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:
    - Higher order terms

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:
    - Higher order terms
    - Better logical reasoning

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:
    - Higher order terms
    - Better logical reasoning
- **Labeled Property Graphs:**

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:
    - Higher order terms
    - Better logical reasoning
- **Labeled Property Graphs:**
  - Simple core idea + expressive

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:
    - Higher order terms
    - Better logical reasoning
- **Labeled Property Graphs:**
  - Simple core idea + expressive
  - Welkin:

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:
    - Higher order terms
    - Better logical reasoning
- **Labeled Property Graphs:**
  - Simple core idea + expressive
  - Welkin:
    - Formal definitions and grammar

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:
    - Higher order terms
    - Better logical reasoning
- **Labeled Property Graphs:**
  - Simple core idea + expressive
  - Welkin:
    - Formal definitions and grammar
    - Logical engine

Comparisons to Existing Solutions (cont')

Comparisons to Existing Solutions (cont')

- **Cyc:**

Comparisons to Existing Solutions (cont')

- **Cyc:**
  - Similarly depend on contexts

Comparisons to Existing Solutions (cont')

- **Cyc:**
  - Similarly depend on contexts
  - Welkin:

Comparisons to Existing Solutions (cont')

- **Cyc:**
  - Similarly depend on contexts
  - Welkin:
    - Flexible data format

Comparisons to Existing Solutions (cont')

- **Cyc:**
  - Similarly depend on contexts
  - Welkin:
    - Flexible data format
    - Open specification

Future Work

Future Work

- Write the Standard

Future Work

- Write the Standard
- Create a Fast Implementation

Future Work

- Write the Standard
- Create a Fast Implementation
- Information Search Programs

Future Work

- Write the Standard
- Create a Fast Implementation
- Information Search Programs
- Revision System

Future Work

- Write the Standard
- Create a Fast Implementation
- Information Search Programs
- Revision System
- Package Management

Future Work

- Write the Standard
- Create a Fast Implementation
- Information Search Programs
- Revision System
- Package Management

Thank you!

Questions?